

Computer Science & IT



Compiler Design



Code Optimisation
Lecture no. 03



By- Ravindra Sir

Recap of Previous Lecture



Topic

Optimization

Topic

Topic

Topics to be Covered



Topic

Optimisation

Topic

Topic

The no of nodes and edges in CFG:

* 1) $i=1$ B_1

* 2) $j=1$ B_2

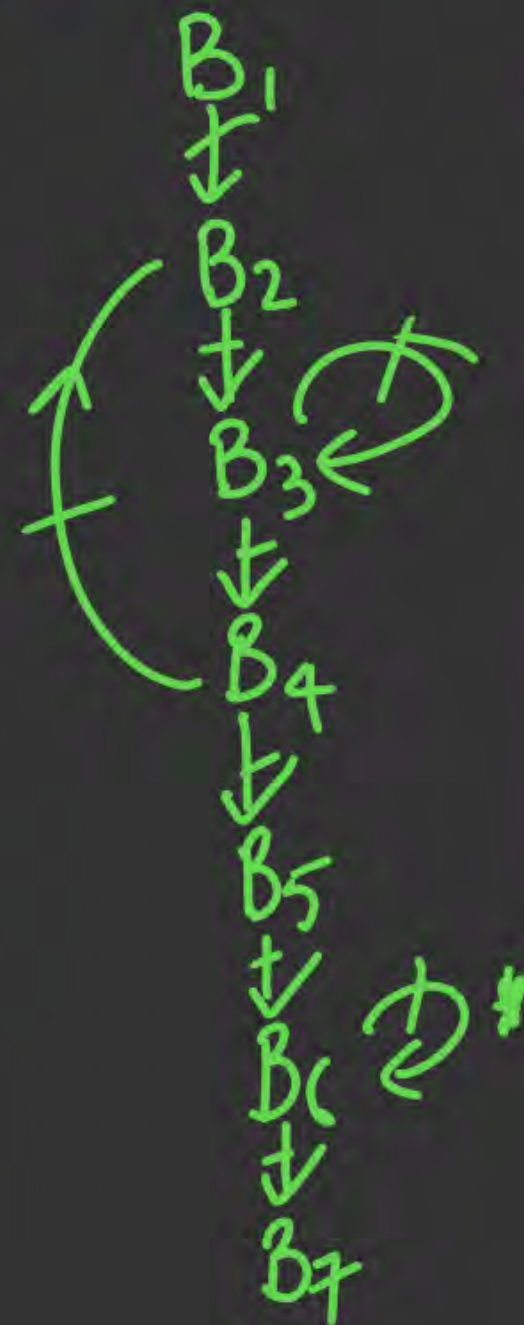
* 3) $t_1 = 2 * i$
4) $t_2 = t_1 + j$
5) $t_3 = 5 * t_2$
6) $t_4 = t_3 + 10$
7) $a[t_4] = 10.5$
8) $j = j + 2$
9) if $(i > 7)$ goto(3) B_3

* 10) $i = i + 2$
11) if $i < 15$ goto(2) B_4

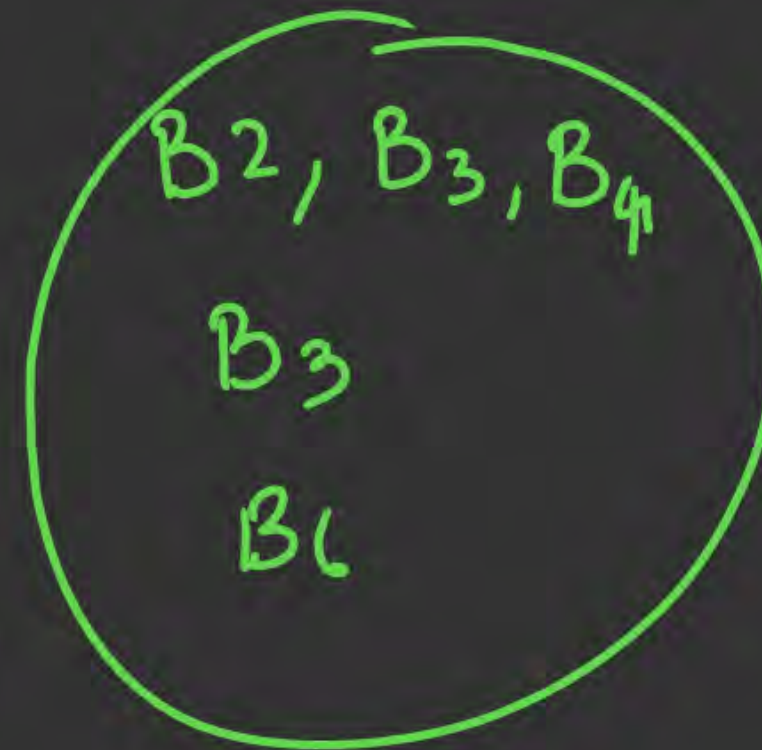
* 12) $i = 5$ B_5

* 13) $t_5 = i * 3$
14) $t_6 = 10 * t_5$
15) $a[t_6] = 5.0$
16) $i = i + 1$
17) if $i < 12$ goto 13 B_6

* 18) Return B_7

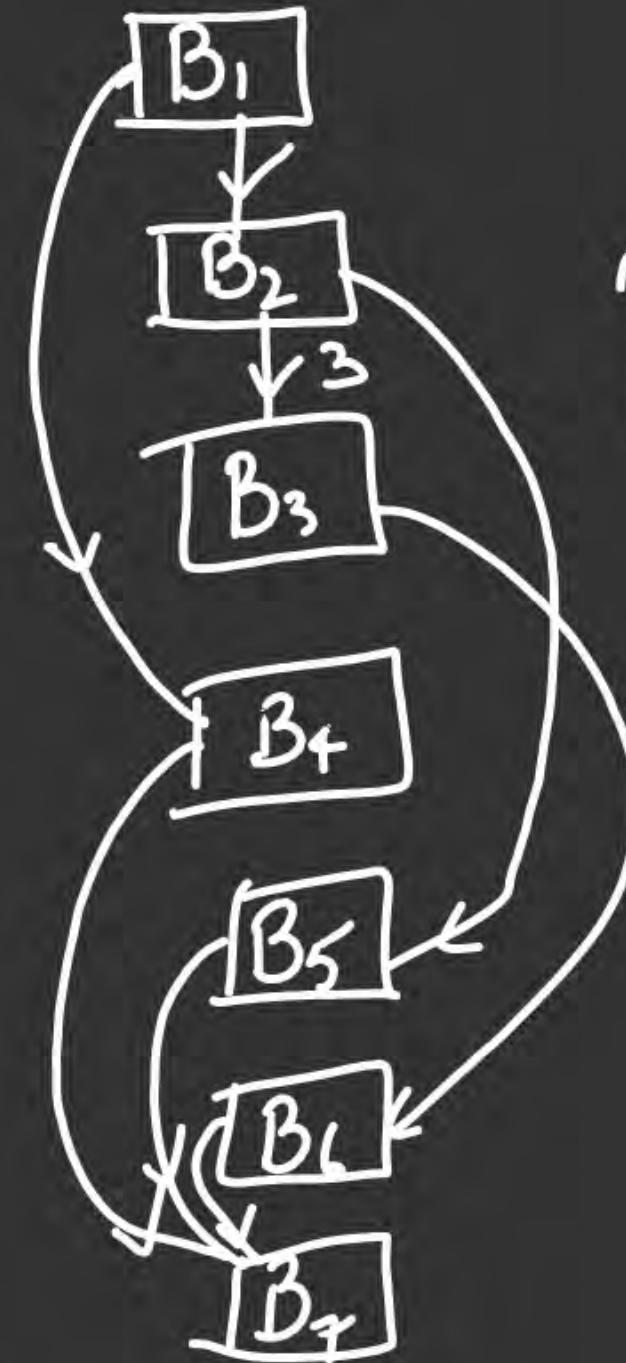
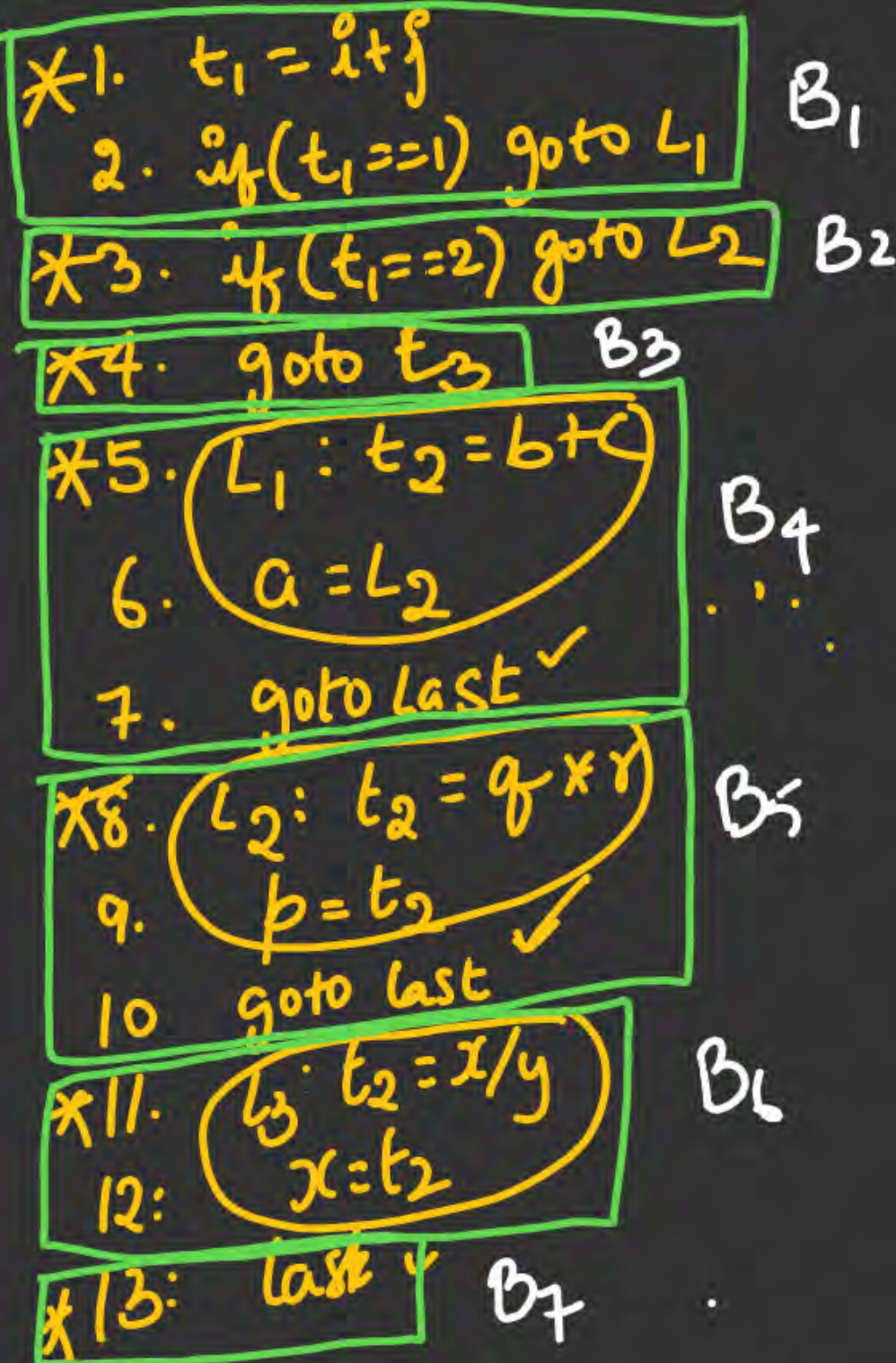


nodes = 7
edges = 9



Find the no of leaders in following code snippet:

```
switch(i+j)
{
  case 1: a=b+c;
          break;
  case 2: p=q*r
          break;
  default: x=x/y
          break;
}
```



NO cycles in CFG
NO loops

Loop optimisations:-



- 1) loop invariant code motion (LICM) of frequency reduction
moves computations that produce the same result on every iteration outside the loop

```
for (int i = 0; i < n; i++)
```

```
{ x = a * b;
```

```
  arr[i] = x + i;
```

```
}
```

$\Rightarrow$

```
x = a * b
```

```
for (i = 0; i < n; i++)
```

```
{ arr[i] = x + i;
```

```
}
```

Loop unrolling:-

Repeats the loop body multiple times to reduce loop control instructions and enable more parallelism

Ex. $\text{for}(\text{int } i=0; i<4; i++)$ $\left\{ \begin{array}{l} a[i] = b[i] + 1; \end{array} \right.$

$$a[0] = b[0] + 1$$

$$a[1] = b[1] + 1$$

$$a[2] = b[2] + 1$$

$$a[3] = b[3] + 1$$

Loop fusion

Combines multiple loops that iterate over the same range into a single loop to reduce overhead and improve data locality

```
for (int i = 0; i < n; i++)
```

```
    a[i] = b[i] + 1;
```

```
for (int i = 0; i < n; i++)
```

```
    c[i] = d[i] - 1;
```

fuse
⇒

```
for (int i = 0; i < n; i++)
{
    a[i] = b[i] + 1;
    c[i] = d[i] - 1;
}
```

3 min

4) Loop fission of distribution:-

Splits a loop into two of more loops to improve parallelism

of cache behaviour

```
for (int i=0; i<n; i++)
```

```
{ a[i] = b[i] + 1;
```

```
  x[i] = y[i] - 1;
```

```
}
```

forall(int i=0; i<n, i++) a[i]=b[i]-1;

forall(int i=0, i<n, i++) x[i]=y[i]-1;

5) Strength Reduction:-

Replaces costly operations (like multiplication) with cheaper ones (like addition, bit shift)

Ex:-

```
for (int i=0; i<n; i++)
```

```
{ x = i * 4;
```

```
}
```

```
for (int i=0; i<n; i++)
```

```
{ x = i << 2;
```

```
}
```

6) loop interchange:-

Swaps the nested loops to improve data access patterns

and cache performance

```
for (int i=0; i<n; i++)
{
    for (int j=0; j<m; j++)
    {
        A[j][i] = 0;
    }
}
```

↓
CO

⇒

```
for (j=0; j<n; j++) {
    for (i=0; i<m; i++)
    {
        A[j][i] = 0;
    }
}
```

machine independent optimizations:-

1) Constant folding

int a = 10 * 5

int b = 100 / 2

int c = 3 + 7 * 2

After Constant folding

int a = 50

int b = 50

int c = 17

2) Constant propagation:-

int x = 5

int y = x + 3

int z = y * 2

after Constant propagation

int x = 5

int y = 8

int z = 10

3) Common Sub-expression elimination:-

int a = (x+y) * 2

int b = (x+y) * 3

after Common Sub-expression elimination

int temp = x+y

int a = temp * 2

int b = temp * 3

u) Dead code elimination:-

int a = ~~10~~ ✓ 30

int b = 20

~~a = 30~~ ✓

int c = a + b

if (false)

{

→ dead code

}

5) algebraic simplification:-

$$\text{int } a = x * 1$$

$$\text{int } b = y + 0$$

$$\text{int } c = z - 0$$

$$\text{int } a = x$$

$$\text{int } b = y$$

$$\text{int } c = z$$

6) Tail recursion elimination

```
int factrial (int n, int result)
```

```
{ if (n==0) return result;
```

```
  return factrial (n-1, result*n); }
```

After tail recursion elimination

```
int factrial (int n)
```

```
{ while (n!=0)
```

```
{ result = result * n;
```

```
  n = n-1;
```

```
  } result result;
```

```
}
```

machine dependent optimizations:-

1) Register allocation
 local allocation
 global allocation

2) use of addressing mode

3) Peep hole optimization

→ CO

3) a) Redundant load and store elimination

$x = y + z$
 $\left\{ \begin{array}{l} \text{mov } y, R_0 \\ \text{add } z, R_0 \\ \text{mov } R_0, x \end{array} \right.$

$a = b + c$
 $d = a + e \Rightarrow$

$\text{mov } b, R_0$ $\text{add } c, R_0$ $\text{mov } R_0, a$ $\text{mov } a, R_0$	$\rightarrow$ Redundant
$\text{add } e, R_0$ $\text{mov } R_0, d$	

3)b) flow of control optimization:-

avoid jumps
on jumps

L: jmp L₁ | L: jmp L₃

L₁: jmp L₂

L₂: jmp L₃

eliminate dead code

#define x 0

if (x)

{

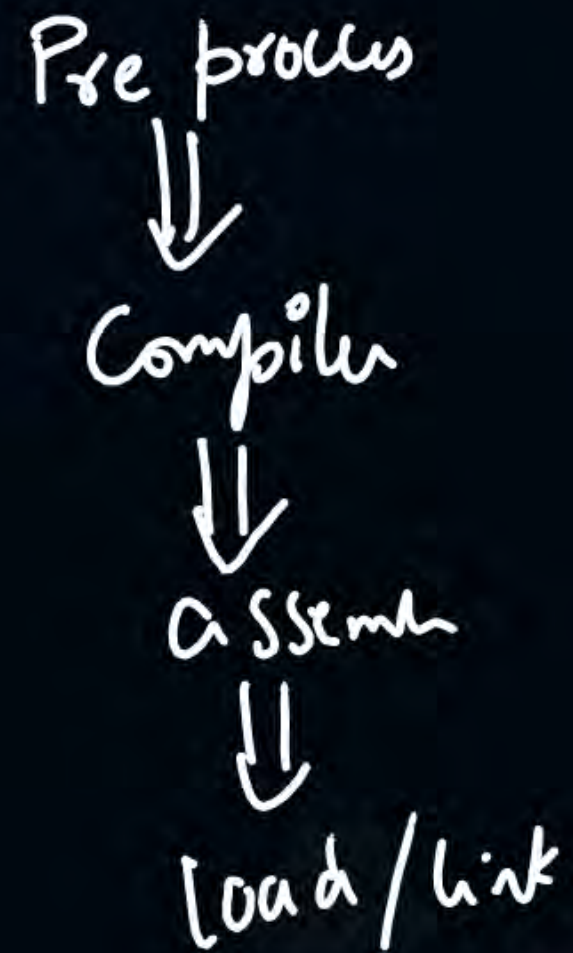
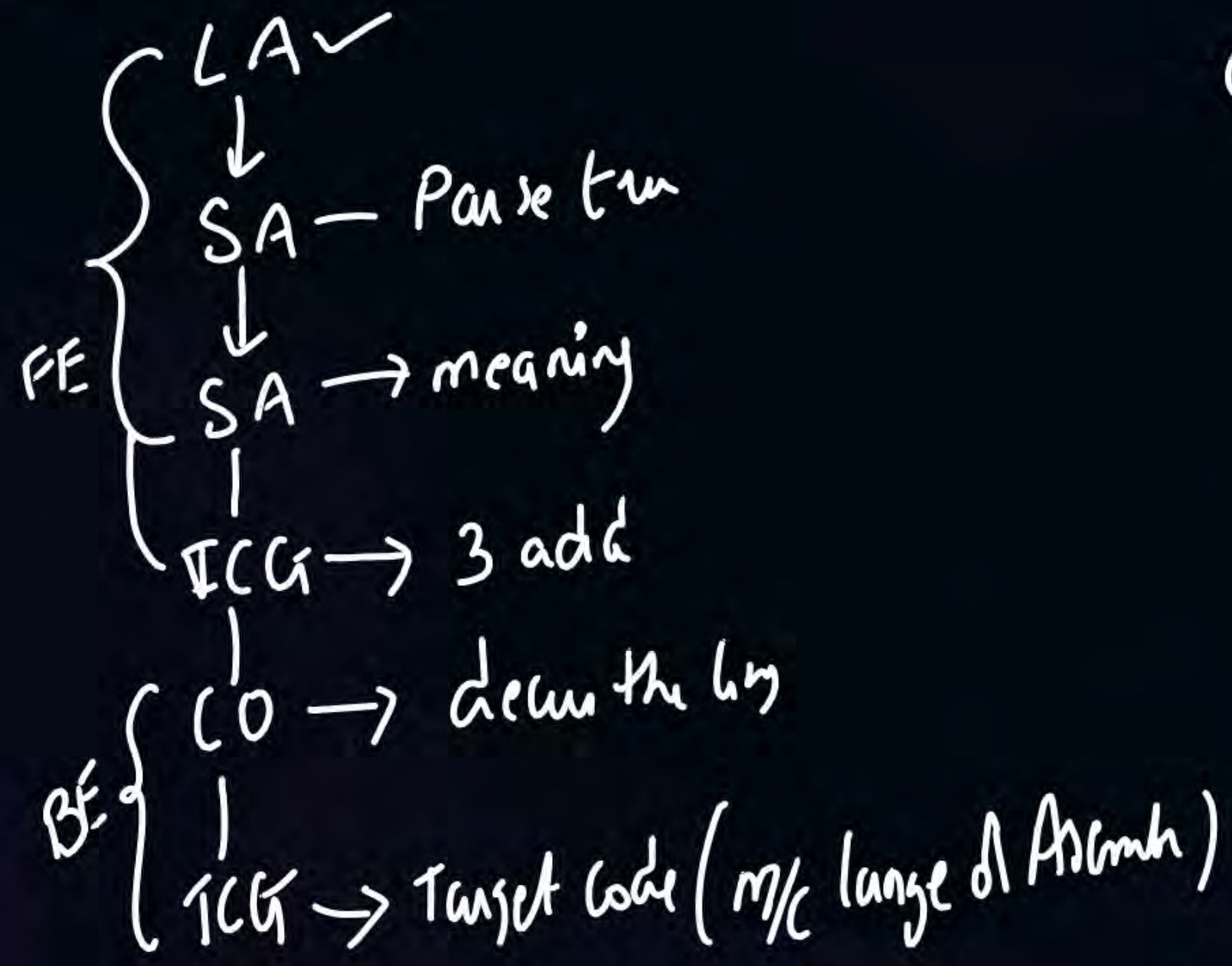
↑
dead code

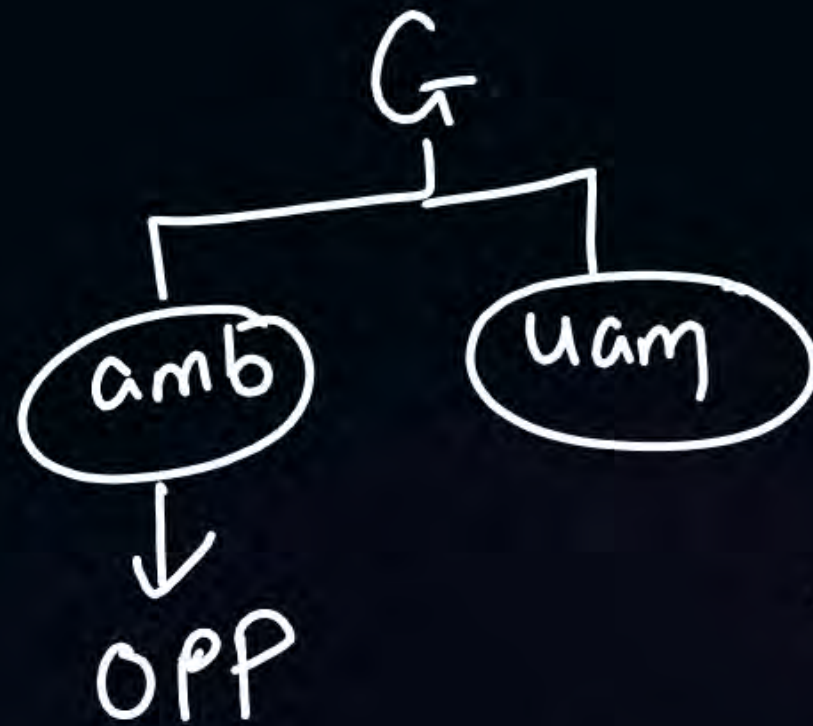
}

c) use of m/c idioms - instruction selection:

$i = i + 1$	<pre> mov R0, i add R0, 1 mov i, R0 </pre> } <u><u>inc i</u></u>
-------------	---

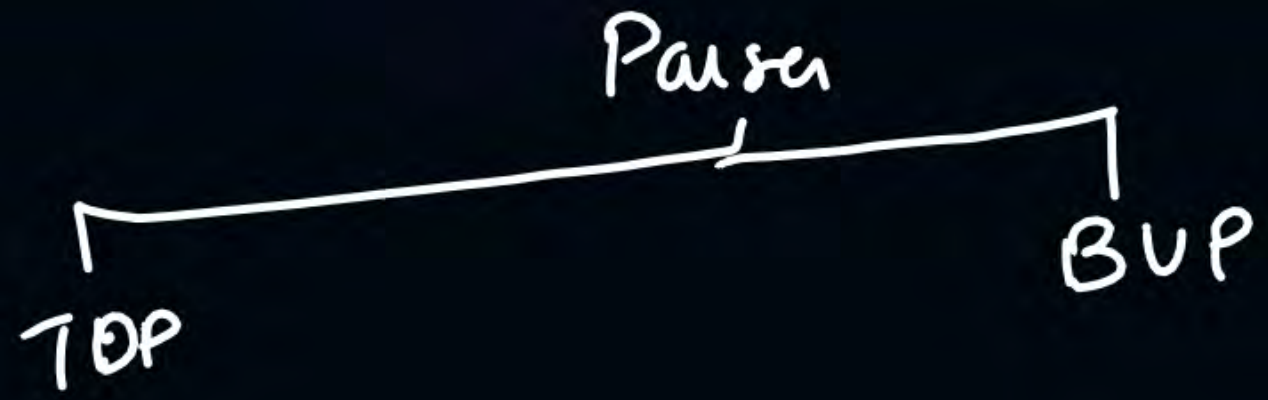
Revision - 15

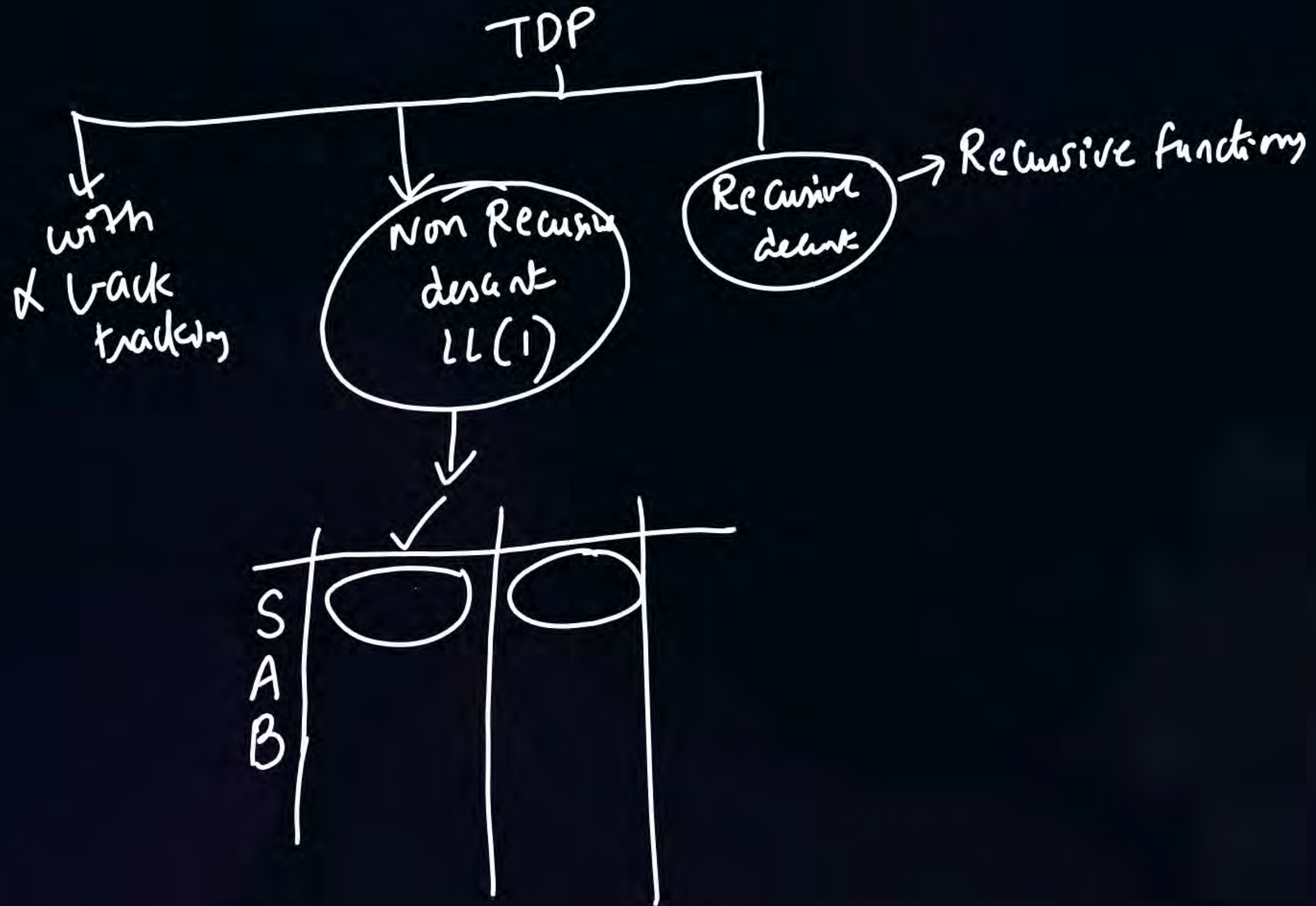


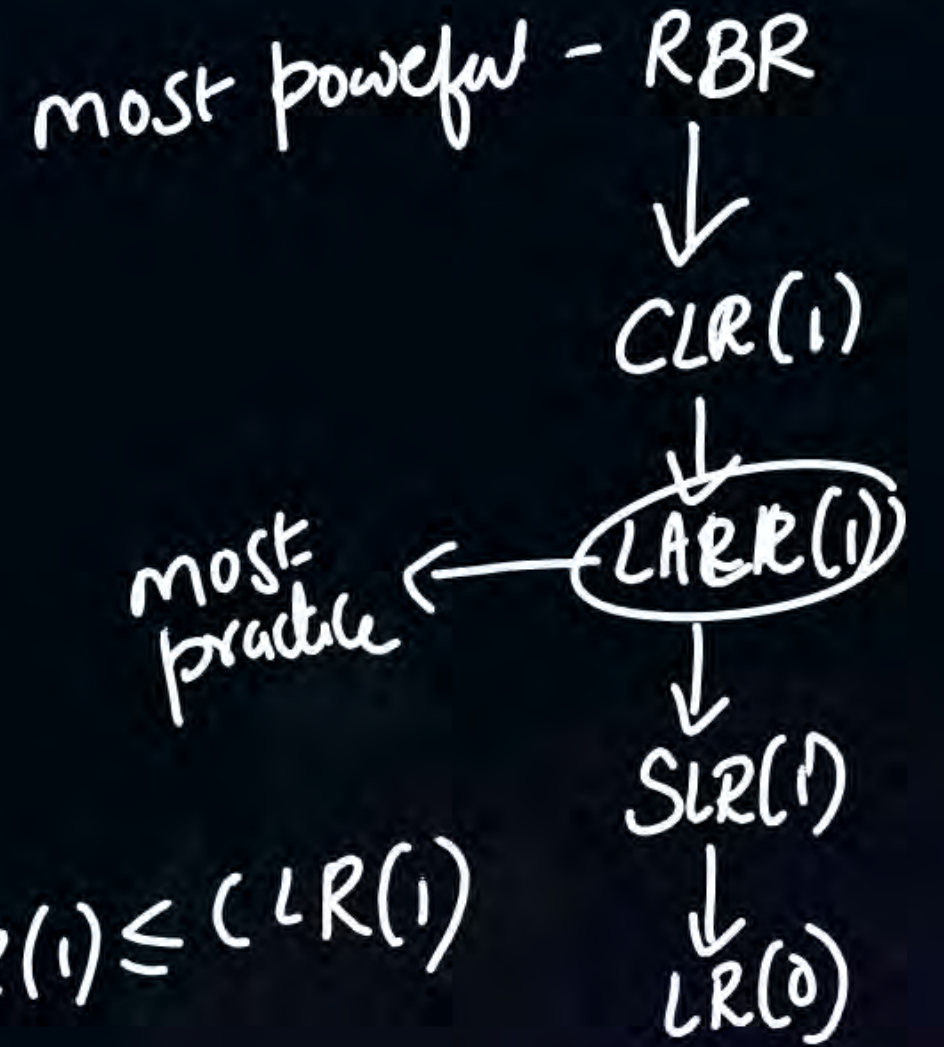
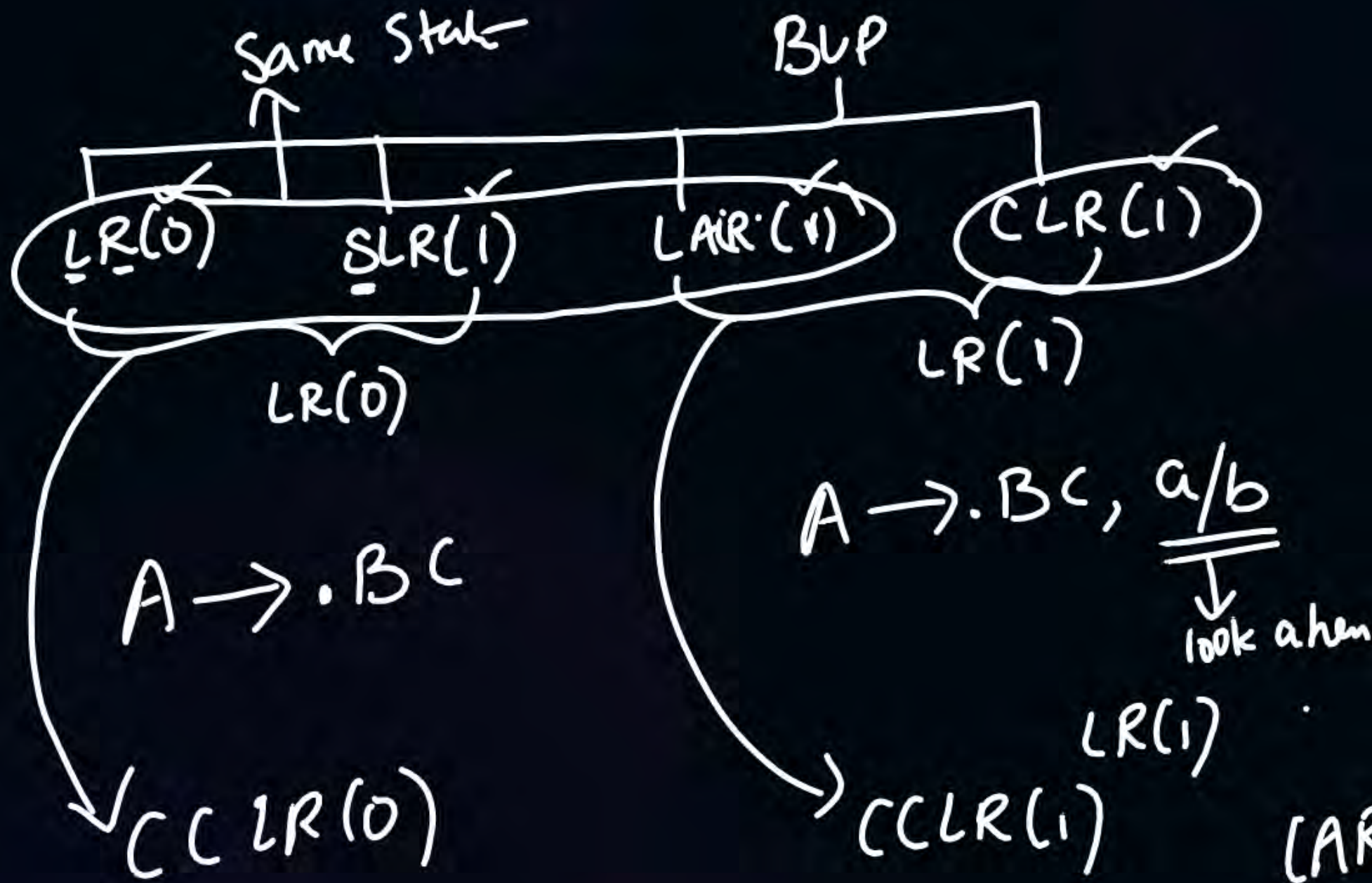


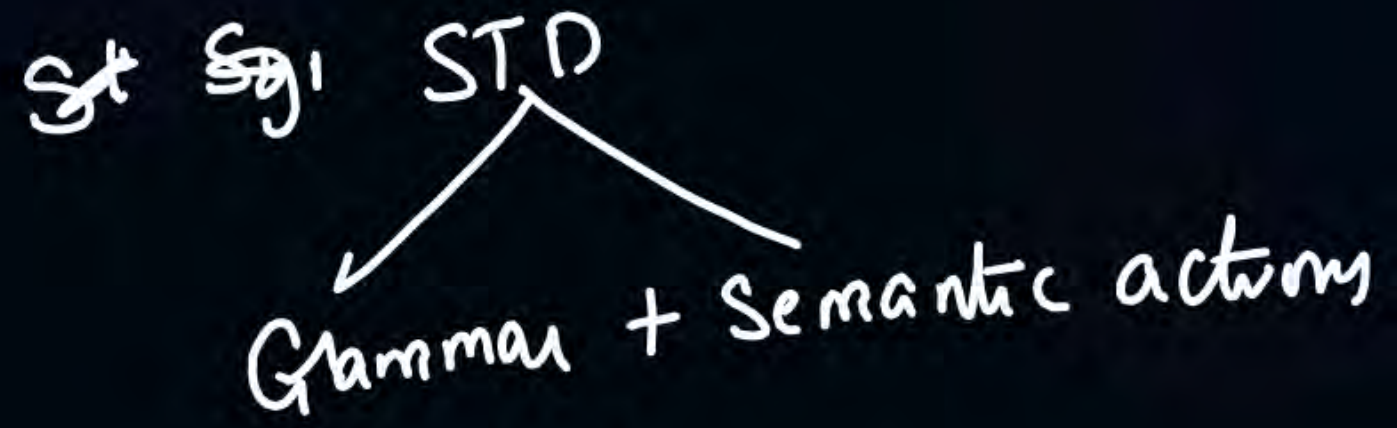
Top should have higher



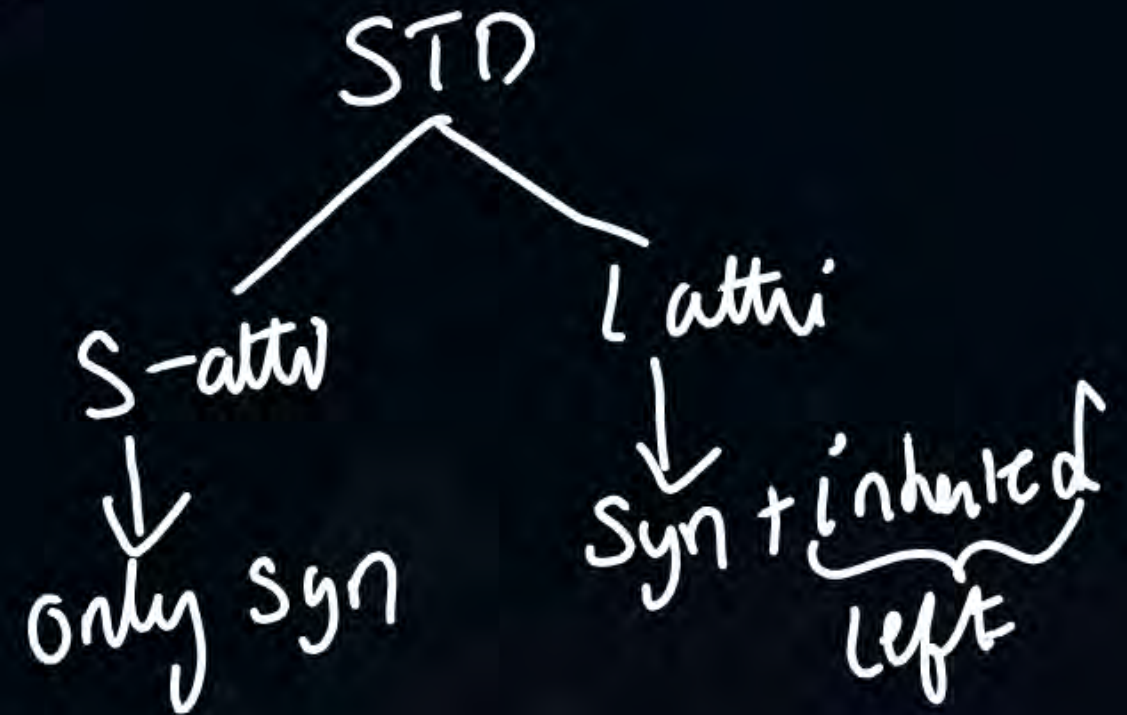




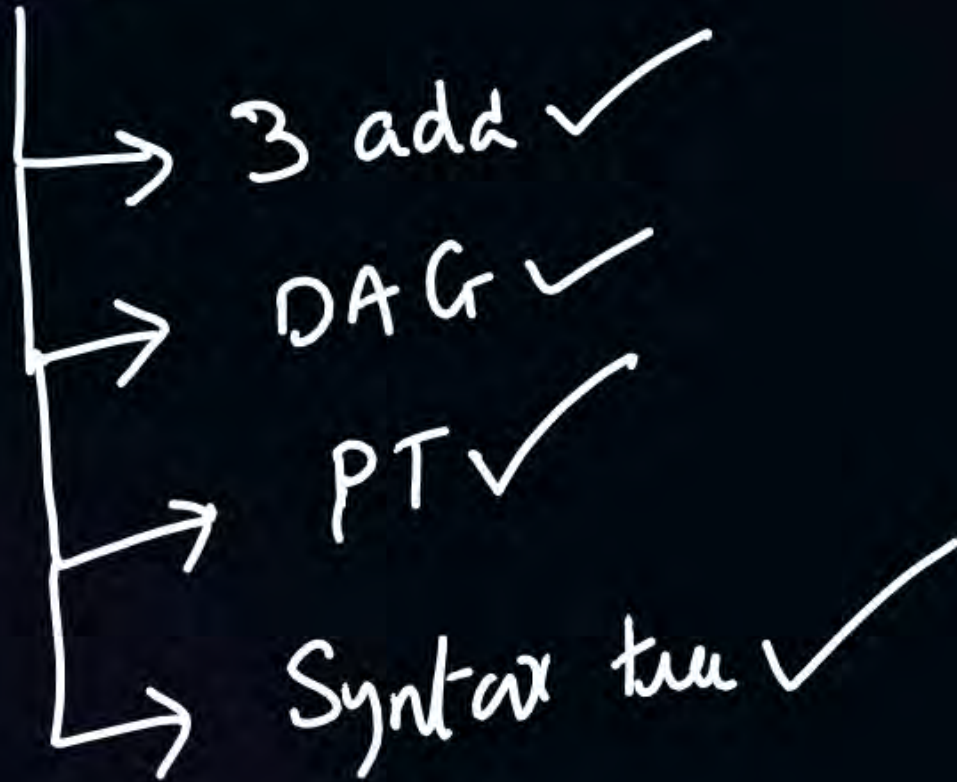




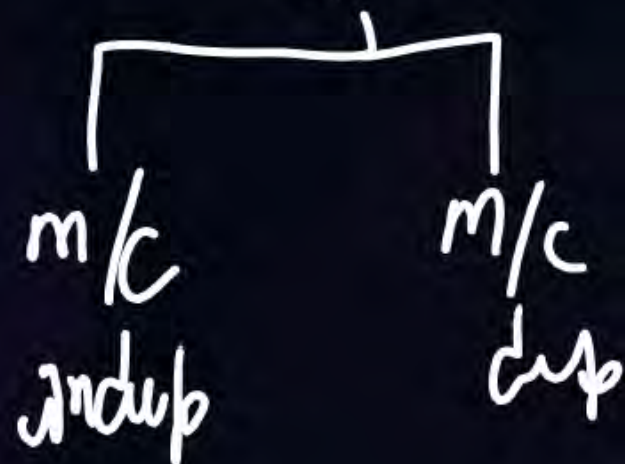
- 1) DAG
- 2) 3 add
- 3) Deriving $\rightarrow$ decl
- 4) Type info
- 5) Exp evaluation



ICG



CO





THANK - YOU